



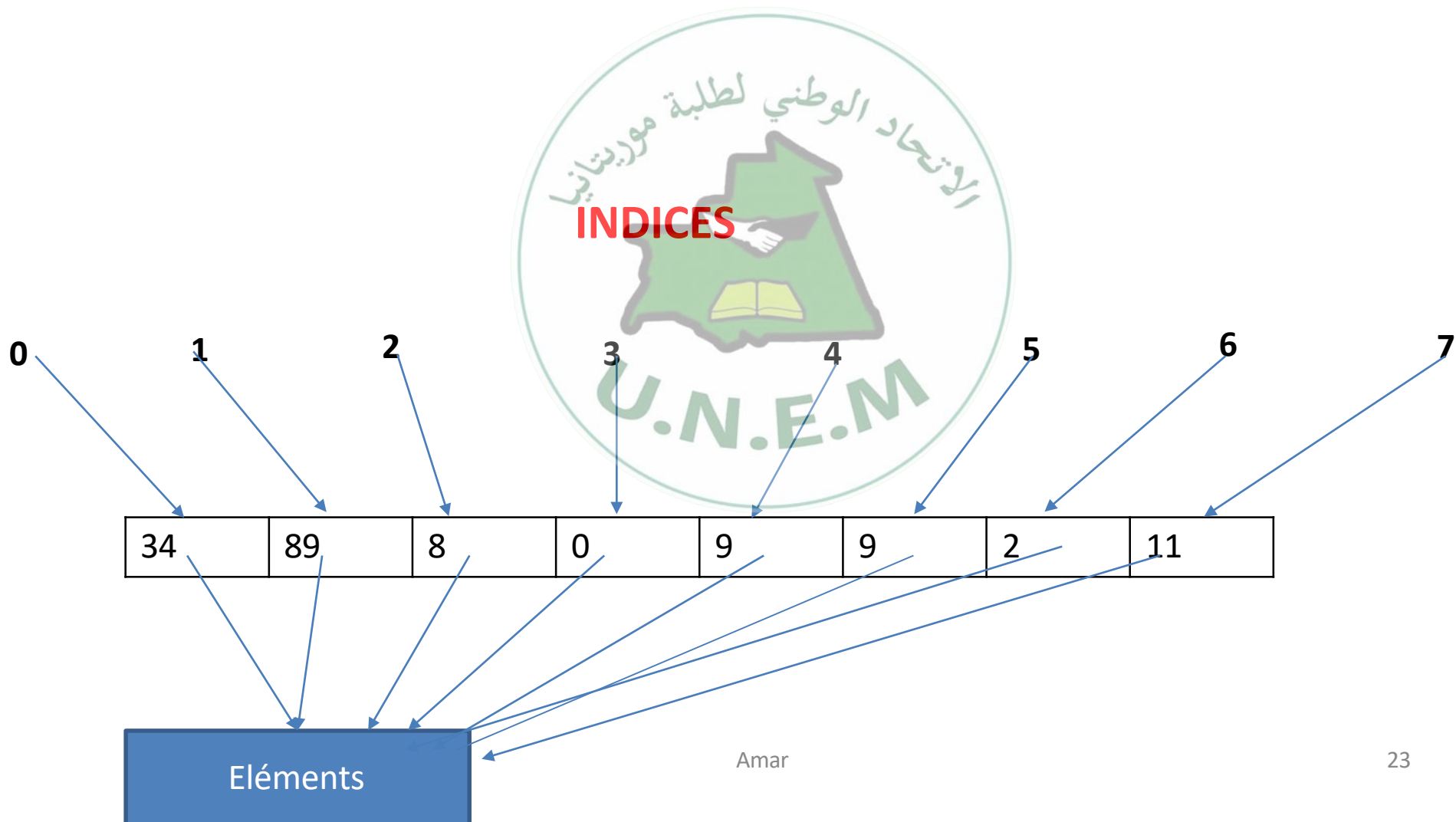
Chapitre 3 : Listes, chaines, tuples et dictionnaires





Listes

Une **liste** est une suite ordonnée d'objets (valeurs), pouvant être de type différent.



Listes

Notation d'une liste

[1,2,3,4] # énumération des objets entre []

[[1,2],[1],1] # liste dont les deux premiers termes sont des listes

[] # liste vide

Les éléments d'une liste peuvent éventuellement être eux- mêmes des listes



Listes

L'accès à un élément d'une liste est direct. **L'indexation** des éléments commence à 0.

Accès aux éléments d'une liste par indexation positive:

```
>>> li = [1,2,3]
```

```
>>> li[0]
```

```
1
```

```
>>> li[2]
```

```
3
```

```
>>> li[3]
```

```
Traceback (most recent call last):
```

```
  File "<stdin>", line 1, in <module>
```

```
IndexError: list index out of range
```

Amar



- On peut aussi accéder aux éléments par **indexation négative**. Le dernier élément de la liste est alors numéroté **-1**. Avec la liste de l'exemple précédent, on obtient par exemple :

```
>>> li[-1]
```

```
3
```

```
>>> li[-3]
```

```
1
```

```
>>> li[-4]
```

```
Traceback (most recent call last):
```

```
File "<stdin>", line 1, in <module>
```

```
IndexError: list index out of range
```

On indique par un **(m)** le fait que la méthode modifie directement la liste sur laquelle elle agit. Les autres retournent le résultat en sortie de fonction.

Opérations et méthodes applicables à une liste:

<code>len(L)</code>	# longueur (nombre d'éléments) de L
<code>L1 + L2</code>	# concaténation des listes
<code>n * L</code>	# pour n entier: concaténation répétée de L avec elle-même
<code>L.append(a)</code>	# ajout de l'objet a en fin de liste (m)
<code>L.insert(i,a)</code>	# insertion de l'objet a en position i (m)
<code>L.remove(a)</code>	# retrait de la première occurrence de a (m)
<code>L.pop(i)</code>	# retrait et renvoi de l'élément d'indice i (m) # par défaut, si i non précisé: dernier élément
<code>L.index(a)</code>	# position de la première occurrence de a # <code>ValueError</code> si a n'est pas dans la liste
<code>L.count(a)</code>	# nombre d'occurrences de a dans la liste
<code>a in L</code>	# teste l'appartenance de a à L
<code>L.copy()</code>	# copie simple de L # attention aux problèmes de dépendance des attributs
<code>L.reverse()</code>	# retourne la liste (inversion des indexations) (m)
<code>L.sort()</code>	# trie la liste dans l'ordre croissant (m)

Exemple



```
# Déclarer les listes
```

```
ISCAE = ['DI', 'IG', 'RT', 'SAE', 'BA', 'FC', 'GRH', 'TCM']
```

```
MED = ['BA', 'FC', 'GRH', 'TCM']
```

```
MQI = ['DI', 'IG', 'RT', 'SAE']
```

```
# 1. append
```

```
ISCAE.append('NOUVEAU')
```

```
print(ISCAE) # ['DI', 'IG', 'RT', 'SAE', 'BA', 'FC', 'GRH', 'TCM', 'NOUVEAU']
```

```
# 2. insert
```

```
ISCAE.insert(2, 'MGT')
```

```
print(ISCAE) # ['DI', 'IG', 'MGT', 'RT', 'SAE', 'BA', 'FC', 'GRH', 'TCM', 'NOUVEAU']
```

```
# 3. index
```

```
indice = ISCAE.index('GRH')
```

```
print(indice) # 7
```



Exemple



```
# count  
count_BA = ISCAE.count('BA')
```

```
print(count_BA) # 1
```

```
# 5. sort
```

```
ISCAE.sort()
```

```
print(ISCAE) # ['BA', 'DI', 'FC', 'GRH', 'IG', 'MGT', 'NOUVEAU', 'RT', 'SAE', 'TCM']
```

```
# 6. reverse
```

```
ISCAE.reverse()
```

```
print(ISCAE) # ['TCM', 'SAE', 'RT', 'NOUVEAU', 'MGT', 'IG', 'GRH', 'FC', 'DI', 'BA']
```

```
# 7. clear
```

```
ISCAE.clear()
```

```
print(ISCAE) # []
```





Exemple

Afficher les éléments de la liste ISCAE suivante:



ISCAE = ['DI', 'IG', 'RT', 'SAE', 'BA', 'FC', 'GRH', 'TCM']



append() et extend()

```
L = [1,3,7]
```

```
L.append([12,7,9])
```

```
print( L)
```

```
[1, 3, 7, [12, 7, 9]]
```

```
L = [1,3,7]
```

```
L.extend([12,7,9])
```

```
print (L)
```

```
[1, 3, 7, 12, 7, 9]
```



Exemple

Définir les listes suivantes:

$L1 = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]$

$L2 = [0, 2, 4, 6, \dots, 16]$

$L3 = [0, 3, 6, 9, 12, \dots, 30]$





Exercices



1. Stocker 10 entiers donnés par l'utilisateur dans une liste?
2. Max d'une liste?
3. Moyenne d'une liste?
4. Rechercher un élément dans une liste



Slicing

- Il s'agit simplement de l'extraction d'une tranche de la liste, en précisant un indice initial et un indice final.

Technique de slicing:

`L[i:j]` # Extraction de la tranche `[L[i], ... , L[j-1]]`

`L[i:j:p]` # De même de p en p à partir de `L[i]`, tant que $i+k*p < j$



- Si le premier indice est omis, il est pris égal à 0 par défaut.
- • Si le deuxième indice est omis, il est pris égal à la longueur de la liste par défaut (on extrait la tranche finale)
- • Si le troisième indice est omis, il est pris égal à 1 par défaut

Technique de slicing:

`L[i:j]` # Extraction de la tranche `[L[i], ... , L[j-1]]`

`L[i:j:p]` # De même de p en p à partir de `L[i]`, tant que `i+k*p < j`



Slicing

Liste initiale

```
ISCAE = ['DI', 'IG', 'RT', 'SAE', 'BA', 'FC', 'GRH', 'TCM']
```

1. Découper une sous-liste (index 2 à 5 exclu)

```
print(ISCAE[2:5]) # ['RT', 'SAE', 'BA']
```

2. Découper les éléments jusqu'à l'index 4 (exclu)

```
print(ISCAE[:4]) # ['DI', 'IG', 'RT', 'SAE']
```

3. Découper les éléments à partir de l'index 4

```
print(ISCAE[4:]) # ['BA', 'FC', 'GRH', 'TCM']
```

4. Prendre un élément sur deux

```
print(ISCAE[::2]) # ['DI', 'RT', 'BA', 'GRH']
```

5. Inverser la liste

```
print(ISCAE[::-1]) # ['TCM', 'GRH', 'FC', 'BA', 'SAE', 'RT', 'IG', 'DI']
```



Slicing

Modifier une partie de la liste avec slicing

```
ISCAE[2:5] = ['MGT', 'MATH']
```

```
print(ISCAE) # ['DI', 'IG', 'MGT', 'MATH', 'BA', 'FC', 'GRH', 'TCM']
```

7. Supprimer une partie de la liste avec slicing

```
ISCAE[3:6] = []
```

```
print(ISCAE) # ['DI', 'IG', 'MGT', 'MATH', 'TCM']
```

8. Ajouter une partie à la liste avec slicing

```
ISCAE[2:3] = ['NEW', 'ITEM', 'ADDED']
```

```
print(ISCAE) # ['DI', 'IG', 'NEW', 'ITEM', 'ADDED', 'MATH', 'TCM']
```

9. Travailler avec une sous-liste avec des indices négatifs

```
print(ISCAE[-3:]) # ['MATH', 'TCM']
```

10. Utiliser un slicing avec un step négatif

```
print(ISCAE[::-2]) # ['TCM', 'MATH', 'NEW', 'IG']
```

Slicing



```
>>> M = [0,1,2,3,4,5,6,7,8,9,10]
```

```
>>> M[3:6]
```

```
[3, 4, 5]
```

```
>>> M[2:8:2]
```

```
[2, 4, 6]
```

```
>>> M[:3]
```

```
[0, 1, 2]
```

```
>>> M[3::3]
```

```
[3, 6, 9]
```

```
>>> M[::5]
```

```
[0, 5, 10]
```

```
>>> M[:2:4]
```

```
[0]
```

```
>>> M[:5:4]
```

```
[0, 4]
```

```
>>> M[-3:-1]
```

```
[8, 9]
```

```
>>> M[2:6:-3]
```

```
[]
```

```
>>> M[6:2:-3]
```

```
[6, 3]
```





listes en compréhension

- Les compréhensions de listes fournissent un moyen de construire des listes de manière très concise.
- Une compréhension de liste consiste à placer entre crochets une expression suivie par une clause **for** puis par **zéro ou plus clauses for ou if**



listes en compréhension

- Par exemple, supposons que l'on veuille créer une liste de carrés, comme :

```
>>> carre = []  
>>> for x in range(10):  
...     carre.append(x**2)  
...  
>>> carre  
[0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
```

- ou, de manière équivalente :

```
carre = [x**2 for x in range(10)]
```

- qui est plus court et lisible



liste par compréhension

- `>>> v = [-4, -2, 0, 2, 4]`
- `>>> L1=[x*2 for x in v]`
- `>>> L2=[x+y for x in [1,2,3] for y in [3,1,4] if x != y]`





Exemple

- Définir les listes suivantes en utilisant liste par compréhension:

- $L1 = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]$

- $L2 = [0, 2, 4, 6, \dots, 16]$



- Liste des nombres de 1 à 10 divisibles par 3
- Liste des cubes des nombres impairs de 1 à 20



Les chaînes de caractères

Python, les chaînes de caractères (ou strings) sont des séquences **immuables** de caractères utilisées pour représenter du texte.

```
#Déclaration
```

```
chaîne1 = 'Bonjour'
```

```
chaîne2 = "Python est génial"
```

```
#chaîne_multiligne
```

```
chaîne_multiligne = """Ceci est une chaîne  
qui s'étend sur  
plusieurs lignes."""
```

```
#Concaténation de chaînes
```

```
chaîne1 = "Bonjour"
```

```
chaîne2 = " tout le monde"
```

```
resultat = chaîne1 + chaîne2 # "Bonjour tout le monde"
```

- L'accès à un caractère d'une chaîne se fait par **indexation** de la même manière que pour les listes. Le premier caractère est **indexé par 0**.
- Les **indexations négatives** sont possibles dans la même mesure que pour les listes.



Accès aux éléments d'une chaîne par indexation positive:

```
>>> ch = 'abcd'
```

```
>>> ch[2]
```

```
'c'
```

```
>>> ch[-1]
```

```
'd'
```

Les slicings peuvent se faire également sur les chaînes de caractères, de la même manière que pour les listes

```
>>> ch = 'azertyuiopqsdghjklmwxcvbn'
```

```
>>> ch[3:12]
```

```
'rtyuiopqs'
```

```
>>> ch[20:10:-1]
```



Une liste des opérations et méthodes les plus utiles.

Opérations et méthodes sur les chaînes de caractères:

<code>len(c)</code>	# longueur (nombre de caractères)
<code>print(c)</code>	# affichage à l'écran de la chaîne c
<code>c + d</code>	# concaténation des chaînes
<code>n * c</code>	# concaténation répétée n fois
<code>d in c</code>	# teste si d est sous-chaîne de c
<code>c.count(d)</code>	# nombre d'occurrences
<code>c.find(d)</code>	# indice de la première occurrence de d # renvoie -1 si pas d'occurrence
<code>c.index(d)</code>	# indice de la première occurrence de d # erreur si pas d'occurrence
<code>c.rfind(d)</code>	# de même que find pour la dernière occurrence
<code>c.rindex(d)</code>	# de même que index pour la dernière occurrence
<code>c.replace(d,e)</code>	# remplace toutes les sous-chaînes d par e

Une liste des opérations et méthodes les plus utiles.



Déclaration de différentes chaînes de texte

```
texte = "Bonjour tout le monde"
```

```
texte2 = " ISCAE"
```

```
texte3 = "###Python###"
```

```
# Affiche 'texte' en majuscules
```

```
print(texte.upper()) # "BONJOUR TOUT LE MONDE"
```

```
# Affiche 'texte' en minuscules
```

```
print(texte.lower()) # "bonjour tout le monde"
```

```
# Supprime les espaces au début et à la fin de 'texte2' et affiche le résultat
```

```
print(texte2.strip())
```

```
# Supprime les caractères '#' au début et à la fin de 'texte3' et affiche le résultat
```

```
print(texte3.strip('#')) # "Python"
```

```
# Vérifie si 'texte' commence par "Bonjour" et affiche True ou False
```

```
print(texte.startswith("Bonjour")) # True
```

```
# Vérifie si 'texte' se termine par "mondee" et affiche True ou False
```

```
print(texte.endswith("mondee")) # False
```



Conversion

```
a = "ISCAE"
```

```
l = list(a)
```

```
print(l)
```

```
['I', 'S', 'C', 'A', 'E']
```



split()

Permet de diviser une chaîne en une liste des sous chaînes selon un séparateur.

```
ch="Welcome to ISCAE"  
res=ch.split()  
print(res)
```

```
['Welcome', 'to', 'ISCAE']
```

```
ch="Nom: Ahmed;Age: 22; Note: 17;Institut: ISCAE"  
res=ch.split(";")  
print(res)
```

```
['Nom: Ahmed', 'Age: 22', ' Note: 17', 'Institut: ISCAE']
```




Méthodes utiles

La méthode **join()** est une méthode de chaîne de caractères (string) qui permet de concaténer les éléments d'une séquence (liste, tuple, etc.) en insérant un séparateur entre eux.

```
"séparateur".join(itérable)
```

EXEMPLE

```
liste_mots = ["Bonjour", "tout", "le", "monde"]  
phrase = " ".join(liste_mots)  
print(phrase)
```

Il donne:

Bonjour tout le monde

isdigit(): Vérifie si tous les caractères de la chaîne sont des chiffres.

```
text = "12345"  
print(text.isdigit()) # Résultat : True
```

isalpha() : Vérifie si tous les caractères de la chaîne sont des lettres alphabétiques.

```
text = "Bonjour"  
print(text.isalpha()) # Résultat : True
```




Les tuples

Les **tuples** en Python sont des séquences ordonnées d'éléments, similaires aux listes, mais immuables. Cela signifie que vous ne pouvez ni modifier, ni ajouter, ni supprimer des éléments une fois le tuple créé.

```
#Déclaration
tuple1 = (1, 2, 3)
tuple2 = ("Python", 3, 14, True)
tuple3 = 1, 2, 3 # Tuple sans parenthèses
#Accès aux éléments
tuple1 = (10, 20, 30, 40)
premier_element = tuple1[0] # 10
dernier_element = tuple1[-1] # 40 (indexation négative)
#Immuabilité
tuple1 = (1, 2, 3)
tuple1[0] = 10 # Provoque une erreur TypeError
```

Il est aussi possible de concaténer, répliquer et d'extraire des tranches au niveau des **tuples**.



Les ensembles

Les ensembles (ou sets) sont des collections non ordonnées d'éléments uniques. Cela signifie que chaque élément dans un ensemble est présent une seule fois, sans duplication, ce qui en fait une structure idéale pour éliminer automatiquement les doublons.

```
#Déclaration
ensemble1 = {1, 2, 3}
ensemble2 = set([4, 5, 6]) # Création à partir d'une liste
ensemble_vide = set() # attention !
ensemble = {1, 2, 2, 3}
print(ensemble) # Affiche : {1, 2, 3}
ensemble = {1, 2, 3}
ensemble.add(4) #Ajout
print(ensemble) # {1, 2, 3, 4}
ensemble.remove(2) # Supprime 2, provoque une erreur si l'élément n'existe pas
ensemble.discard(5) # Ne provoque pas d'erreur si l'élément n'existe pas
```

Comme ce sont des ensembles, on peut utiliser les opérations ensemblistes comme l'union, la différence, etc.

Exercice



Exercice 1

Soit la liste: $L = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]$

A partir de L:

- Ajouter la valeur 1 à chacun de ses éléments.
- Ajouter la valeur 11 à la fin de la liste.
- Ajouter les valeurs 12 et 13 à la fin de la liste.
- Afficher le premier élément, les deux premiers éléments, le dernier élément, les deux derniers éléments.
- Construire la liste "paires" qui contient les nombres paires de L et la liste "impaire" qui contient les nombres "impaires" de L.
- boucle for et liste compréhension
- Ajouter la valeur 3.5 entre 3 et 4.
- Supprimer la valeur 3.5.
- Inverser l'ordre des éléments de L.
- Demander à l'utilisateur de fournir un nombre au hasard et dire si ce nombre est présent dans L.



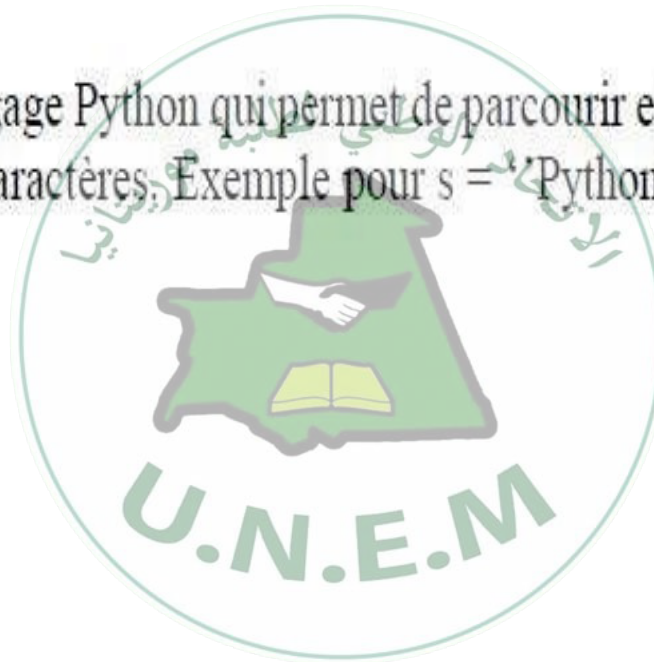
Exercice



Exercice 2

Ecrire un programme en langage Python qui permet de parcourir et afficher les caractères d'une variable du type chaîne de caractères. Exemple pour `s = 'Python'`, le programme affiche les caractères :

P
y
t
h
O
n





Dictionnaires





Dictionnaires

Les **dictionnaires** sont des **collections non ordonnées** d'éléments. Cela signifie qu'ils ne possèdent **pas d'indices** comme les listes, et donc **aucune notion d'ordre** des éléments.

L'accès aux **valeurs** se fait à l'aide de **clés**, et non pas d'indices numériques.

Exemple :

```
>>> p1 = { } ou bien p1=dict() #dictionnaire vide
```

```
>>> p1 [" nom "] = " Aly "
```

```
>>> p1 [" taille "] = 1.7
```

```
>>> p1 [" poids "] = 80
```

```
>>> print(p1)
```

```
{'nom ': ' Aly ', 'taille ': 1,7 , 'poids ': 80}
```



Dictionnaires

- On peut aussi initialiser toutes les clés et les valeurs d'un dictionnaire en une seule opération :

```
>>> p2 = {" nom ": " Fatma ", " poids ": 70 , " taille ": 1.75 }
```

- On peut ajouter une clé et une valeur supplémentaire :

```
>>> p2 [" age "] = 15
```

- Pour récupérer la valeur associée à une clé donnée, il suffit d'utiliser la syntaxe suivante `dictionnaire["cle"]`.

Par exemple :

```
>>> p2 [" taille "] ➔ 1.75
```




Dictionnaires

- Il est possible d'obtenir toutes les valeurs d'un dictionnaire à partir de ses clés :

```
>>> p2 = {" nom ": " Fatma ", " poids ": 70 , " taille ": 1.75}
```

```
>>> for i in p2 :
```

```
    print ( i , p2 [ i ])
```

```
nom      Fatma
poids    70
taille   1.75
```



Méthodes .keys(), .values() et .items()

- Les méthodes .keys() et .values() renvoient, les clés et les valeurs d'un dictionnaire :

```
>>> p2 . keys ()
```

```
dict_keys(['nom', 'poids', 'taille'])
```

```
>>> p2 . values ()
```

```
dict_values(['Fatma', 70, 1.75])
```



méthode .items()

- Il existe la méthode .items() qui renvoie un nouvel objet dict_items :

```
>>> dico = {0: " t", 1: "o", 2: "t", 3: "o "}
>>> dico . items ()
dict_items([(0, ' t'), (1, 'o'), (2, 't'), (3, 'o ')])
```

- **Pour récupérer clé-valeur**

```
for i, j in dico.items():
... print ( i, j)
```

0	t
1	o
2	t
3	o



Existence d'une clé

- Pour vérifier si une clé existe dans un dictionnaire, on peut utiliser le test d'appartenance avec l'instruction **in** qui renvoie un booléen :

```
>>> p2 = {" nom ": " Fatma ", " poids ": 70 , " taille ": 1.75}  
>>> if "poids" in p2 :  
    print (" La clé 'poids' existe pour p2 ")
```



Tri par clés

- On peut utiliser la fonction `sorted()` **vue précédemment avec les listes** pour trier un dictionnaire par ses clés :
- `p2 = {" nom ": " Fatma " , " taille ": 1.75 , " poids ": 70 }`
- `sorted (p2 )`
- `['nom ' , 'poids ' , 'taille ']`
- Les clés sont triées ici par ordre alphabétique.



Par exemple si on veut stocker les notes des élèves

```
d1 = {"Aly": 14, "Dia": 5}
```

"Aly" est une clé et 14 est la valeur associée.

Si on veut la note de Dia c'est simple

```
print(d1["Dia"])
```

#Si on veut ajouter un élève c'est simple aussi

```
d1["Brahim"] = 13.5
```

```
print(d1)
```

#Je veux supprimer un élève j'utilise del

```
del d1["Brahim"]
```

```
print(d1)
```

#On veut changer les notes de Dia

```
d1["Dia"] = d1["Dia"] + 6
```

On affiche la taille

```
print(len(d1))
```

#On peut afficher le dictionnaires

```
print(d1)
```





update()

La méthode **update()** des dictionnaires en Python permet de mettre à jour un dictionnaire existant avec les clés et les valeurs d'un autre dictionnaire ou d'une liste de paires clé-valeur.

dictionnaire1.update(dictionnaire2)

La méthode **update()** ajoute toutes les clés et valeurs du dictionnaire **dictionnaire2** au dictionnaire **dictionnaire1**. Si une clé existe déjà dans **dictionnaire1**, la valeur correspondante est remplacée par la nouvelle valeur de **dictionnaire2**.

Exemple



```
# Créer deux dictionnaires
```

```
dict1 = {"a": 1, "b": 2, "c": 3}
```

```
dict2 = {"d": 4, "e": 5}
```

```
# Mettre à jour dict1 avec les clés et valeurs de dict2
```

```
dict1.update(dict2)
```

```
# Afficher le résultat
```

```
print(dict1) # {'a': 1, 'b': 2, 'c': 3, 'd': 4, 'e': 5}
```